

Agentic Ontology of Work (AOW)

**A foundational semantic model for intelligent, autonomous,
and governed enterprise work**

Version 2.0 · September 2026 · Manish Garg, Skan.ai

Licensed [CC-BY 4.0](#). The machine-readable ontology, shapes, schemas, and examples that accompany this paper are licensed Apache 2.0 and published at github.com/Skandotai/agentic-ontology-of-work.

Abstract

Enterprises are putting agents to work faster than they can agree on how to describe that work. "Agent," "task," "workflow," and "orchestration" mean different things to different vendors and teams, and the result is integrations that break at the seams, governance policies that cannot be applied consistently, and audit trails that stop where one system hands off to the next.

The Agentic Ontology of Work (AOW) is a platform-agnostic semantic model for that work. It names twenty-five kinds of things that agentic systems, people, and the governance around them have in common, and thirty-three relationships between them. It follows work from the business Objective it serves, through the Intents, Plans, and Tasks derived from it, to the Actions that Actors (Agents and people alike) take with their Skills, the Results those Actions produce, and the Outcomes those Results add up to. Around that path, it places the things that make autonomy safe to grant: Policy, Confidence, a five-level Assurance Level scale, an independent Guardian, and a Feedback loop into Memory.

Version 2.0 does two things version 1.0 did not. It fills the gaps a close reading of 1.0 exposes, including undefined Actions, Tasks, and autonomy levels, and no place for the people in the loop. And it ships the ontology in the machine-readable formats 1.0 promised: OWL, JSON-LD, SHACL, and JSON Schema, with worked examples and queries that check an explanation can be reconstructed and that no Agent acted beyond the oversight its Assurance Level requires.

1 Introduction

1.1 The rise of the agentic enterprise

The modern enterprise is moving from systems that automate tasks to systems that understand, reason, and act across complex operations. Several developments are converging to drive the shift: multi-agent systems, learned behavior, contextual awareness, structured governance, continuous feedback loops, and observability at enterprise scale.

The capability to build agentic systems is arriving quickly. A shared language for describing them is not. Enterprises lack an agreed way to say what these systems are, how they behave, how they relate to one another and to the people they work alongside, and how they must be governed.

Enterprise computing has been here before. Service-Oriented Architecture (SOA) gave the industry its first widely adopted formalism for services, endpoints, and contracts, and the shared vocabulary mattered as much as the technology. The agentic era needs the equivalent: a common way to describe Intents, Agents, Skills, Policies, Context, Outcomes, Assurance, Memory, and Feedback.

1.2 Why an ontology, not a glossary

A glossary defines words. An ontology is "an explicit specification of a conceptualization" (Gruber, 1993): it names the kinds of things in a domain, the attributes those things have, and the relationships that connect them, precisely enough that software can check whether a description conforms.

Without a shared ontology, enterprises face:

- inconsistent definitions, in which "agent," "workflow," and "task" overlap
- brittle integrations between platforms that model the same work differently
- governance blind spots where no one can say which rules applied to an action
- ambiguous responsibility when work passes between agents and people
- no reliable way to scale autonomous activity

A well-designed ontology anchors governance and compliance, makes behavior explainable, improves interoperability across platforms, reduces ambiguity, and becomes the grammar in which intelligent work is described.

1.3 Goals

AOW aims to:

- establish a platform-agnostic semantic framework for agentic work
- enable interoperability across multi-agent systems, robotic process automation, AI systems, and human workflows
- support governance, auditability, and risk management by design
- provide a foundation that standards bodies and reference architectures can build on
- help enterprises scale intelligent automation safely, with trust and transparency

1.4 Scope and non-goals

AOW describes work: what is being attempted, by whom, under what constraints, with what effect, and what was learned. It deliberately does not specify:

- **how agents are built.** Models, prompts, and frameworks are implementation details. An Agent's `implementation_ref` may point to them.
- **how agents talk to each other.** Protocols such as Agent2Agent (A2A) and the Model Context Protocol (MCP) carry AOW entities; AOW does not redefine them (section 12.6).
- **a policy language.** A Policy's rules may be written in ODRL, Rego, Cedar, DMN, or plain text (section 4).
- **a storage technology.** AOW can be stored in a knowledge graph, a relational database, or a document store (section 12.4).

1.5 What changed from version 1.0

Version 1.0 (January 2026) set out the four-layer model, fifteen canonical entities, the lifecycle, and the validation criteria. Version 2.0 keeps all of that and is backward compatible in meaning: every 1.0 entity still exists and still means what it meant. The main changes are:

- **Eight new classes** fill gaps 1.0 named but did not define: Work Item, Signal, and Observation (Perception); Plan and Task; and Actor, Human Actor, and Role, which make people first-class participants. Orchestrator and Guardian become kinds of Actor.
- **Action is now defined.** 1.0 listed it in the glossary and registry but gave it no section, and said in one place that Skills produce Results and in another that Actions do. In 2.0, a Skill is a capability, an Action is its use, and the Action produces the Result.

- **The Assurance Level scale is defined**, from AL0 (Manual) to AL4 (Autonomous), with a rule for the effective level when an Agent, a Task, and a Skill disagree (section 7).
- **Provenance and Versioning become a facet** of every entity, expressed with W3C PROV-O, rather than two entities of their own (section 5).
- **Every entity has attributes and relationships**. In 1.0, Guardian, Memory, Provenance, and Assurance Level had none, or only one of the two.
- **The ontology ships in machine-readable form**, with validation (section 9).
- **Editorial fixes**: consistent numbering, corrected example schemas, and a more measured claim about what the ontology is.

Appendix B lists every change.

2 Design principles

AOW is built on seven principles. The first six are carried over from version 1.0; the seventh is new.

Principle 1: Platform agnosticism. The ontology describes concepts, not implementations. No vendor-specific models, training mechanisms, or product metaphors are embedded in it.

Principle 2: Hierarchical simplicity with semantic completeness. The ontology must be rich enough to model complex work and simple enough for architects, engineers, risk teams, and business owners to use.

Principle 3: Governed autonomy. Agentic systems operate under constraints, not unchecked freedom. Policy, Assurance Levels, and provenance are first-class.

Principle 4: Observability and explainability. No action happens without visibility, and no decision is made without a traceable rationale.

Principle 5: Contextual intelligence. Actions are always understood in relation to the environment, history, constraints, and intended Outcomes.

Principle 6: Continuous learning. The ontology supports the accumulation of experience and adaptation as environments shift and agents evolve.

Principle 7: Actor neutrality. Work is described the same way whoever performs it. A Task can be assigned to an Agent or a person; an Action has the same shape and leaves the same trail either way. This is what lets work move between people and agents, in either direction, without being re-modeled, and what lets human oversight be recorded rather than assumed.

3 The four-layer stack

Work is not a line; it is a graph connecting Objectives, Intents, Context, constraints, Actors, Actions, Observations, Feedback, and Outcomes. AOW organizes that graph into four layers, each answering a different question.

Layer	Question it answers	Entities
1 Perception	What exists, and what is happening?	Work Item, Signal, Observation
2 Cognition	What should happen, and within what limits?	Objective, Intent, Context, Policy, Plan
3 Execution	Who does the work, and what did it change?	Task, Actor, Agent, Human Actor, Orchestrator, Role, Skill, Action, Result
4 Assurance	Was it safe, did it work, and what did we learn?	Confidence, Assurance Level, Guardian, Outcome, Feedback, Memory, Operational Memory, Knowledge Base

Layer 1: Perception captures the ground truth: the Work Items in flight and the Signals (events, logs, documents, interactions, state changes, metrics) and Observations about them. Version 1.0 named Telemetry Events, Signals, Session State, and Environmental Observations here without defining them. In 2.0, a telemetry event is a Signal, session state is session Context, and an environmental observation is an Observation.

Layer 2: Cognition evaluates goals, constraints, context, and policies to decide what should happen next, and plans how.

Layer 3: Execution is where Actors take on Tasks, invoke Skills through Actions, and produce Results.

Layer 4: Assurance ensures safety, reliability, trust, and improvement: it measures certainty, grades autonomy, oversees behavior, evaluates Outcomes, and carries Feedback into Memory.

The layers describe what an entity is for, not a deployment architecture. One system may span several layers, and entities in every layer carry the provenance facet described in section 5.

4 Canonical entities

This section defines each entity: what it is, why it exists, its attributes, and the relationships it can start. Attribute and relationship keys are shown as they appear in AOW JSON documents (`snake_case`); the RDF property for each is the same name in `lowerCamelCase` in the `aow: namespace` (`goal_statement` becomes `aow:goalStatement`). Every entity also has an `id` and a `type` , and may carry the common attributes `label` , `description` , and the provenance facet (section 5).

Where a relationship is shown with more than one target, any of them is allowed. "Required" means a conformant document must include it.

4.1 Work Item

Layer: Perception · since 2.0 · `aow:WorkItem`

Definition. The unit of business work that the rest of the ontology is about: a claim, an application, an order, a case, a ticket.

Purpose. Gives every Intent, Task, Action, and Observation a concrete subject, so that work can be followed end to end regardless of which systems, agents, or people touched it.

Attribute	Type	Required	Description
<code>item_type</code>	string	yes	The kind of work item, for example "insurance-claim" or "purchase-order".
<code>external_ref</code>	string		Identifier of the item in its system of record.
<code>state</code>	string		Current business state of the item, in the vocabulary of its system of record.
<code>opened_at</code>	datetime		When the item entered the process.
<code>closed_at</code>	datetime		When the item left the process, if it has.

Notes.

- Version 1.0 carried the work item implicitly inside Context (for example, "ctx-claim-84933"). Making it explicit lets two Intents, or an Intent and a human Task, refer to the same claim.

- Corresponds to a case or object in process mining (IEEE 1849 XES and the OCEL 2.0 object-centric event log format).

4.2 Signal

Layer: Perception · since 2.0 · aow:Signal

Definition. A raw, time-stamped trace emitted by a system, a person, or the environment: an event, a log line, a user-interface interaction, a document arriving, a state change, a metric sample.

Purpose. Anchors the ontology in what actually happened, before any interpretation. Signals are the evidence that Observations, Context, and audit trails are built from.

Attribute	Type	Required	Description
signal_type	one of event , log , metric , interaction , document , state-change	yes	The kind of trace.
source	string	yes	The system, sensor, application, or channel that emitted the Signal.
observed_at	datetime	yes	When the Signal occurred.
payload_ref	uri		Where the raw payload is stored, if it is kept.
sensitivity_level	one of public , internal , confidential , restricted		Handling classification of the payload.

Relationships.

- generated_by → Action

Notes.

- Version 1.0 listed Telemetry Events, Signals, Session State, and Environmental Observations as Perception-layer elements without defining them. In 2.0 a Telemetry

Event is a Signal whose signal_type is "event" or "metric"; Session State is Context with context_type "session".

- Actions leave Signals behind. Linking a Signal to the Action that generated it (generated_by) is what makes agent behavior observable with the same tools used to observe human and system work.

4.3 Observation

Layer: Perception · since 2.0 · aow:Observation

Definition. An interpreted fact about a Work Item, Actor, or Action, derived from one or more Signals and carrying a Confidence.

Purpose. Separates what was seen (Signal) from what it was taken to mean (Observation), so that a wrong interpretation can be traced and corrected without doubting the evidence.

Attribute	Type	Required	Description
observed_property	string	yes	What was observed, for example "claim_amount" or "handling_time".
observed_value	json		The observed value, in any JSON form.
observed_at	datetime		When the observation was made.
method	string		How the value was derived, for example "document-extraction" or "rule".

Relationships.

- generated_by → Action
- derived_from → Signal (one or more) (*required*)
- about → Work Item / Actor / Action / Task
- has_confidence → Confidence

Notes.

- "The claim amount is \$38,200" is an Observation; the scanned claim form it was extracted from is a Signal.

4.4 Objective

Layer: Cognition · since 1.0 · aow:Objective

Definition. A business-level goal expressed in strategic or operational terms.

Purpose. Connects agentic activity to specific, measurable enterprise value.

Attribute	Type	Required	Description
kpi_target	string		The measurable target, for example "claims cycle time -20%".
priority	one of low , medium , high , critical		Relative business priority.
time_horizon	string		The period over which the Objective is pursued, for example "FY2027".
description	text	yes	Longer human-readable explanation.

Notes.

- Objective answers why. Intent answers what.

4.5 Intent

Layer: Cognition · since 1.0 · aow:Intent

Definition. A structured, actionable goal derived from an Objective and interpretable by agents, orchestrators, and people.

Purpose. Turns a business goal into something that can be planned, assigned, governed, and checked.

Attribute	Type	Required	Description
<code>goal_statement</code>	text	yes	What the Intent is meant to achieve, stated as an outcome rather than a procedure.
<code>constraints</code>	string-list		Free-text constraints that are not (yet) captured as Policies.
<code>priority</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		Relative priority.
<code>acceptable_risk_band</code>	one of <code>low</code> , <code>medium</code> , <code>high</code>		The level of residual risk the requester accepts.
<code>status</code>	one of <code>proposed</code> , <code>active</code> , <code>fulfilled</code> , <code>abandoned</code>		Where the Intent is in its life.

Relationships.

- `serves_objective` → Objective (*required*)
- `shaped_by` → Context (one or more)
- `concerns` → Work Item (one or more)
- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level (*required*)

Notes.

- Objective answers why. Intent answers what. Plan answers how.

4.6 Context

Layer: Cognition · since 1.0 · `aow:Context`

Definition. The structured situational information that gives relevance to an Intent.

Purpose. Makes the same Intent resolve differently for different situations, and records what an Actor knew when it acted.

Attribute	Type	Required	Description
context_type	one of environmental , procedural , data , historical , session		The kind of situational information.
variables	json		The situational values themselves.
sensitivity_level	one of public , internal , confidential , restricted		Handling classification.
valid_until	datetime		When the Context should be considered stale.

Relationships.

- draws_on → Memory / Observation (one or more)

Notes.

- Context types: environmental, procedural, data, historical, session.

4.7 Policy

Layer: Cognition · since 1.0 · aow:Policy

Definition. Declarative constraints defining permissible, required, or restricted behavior.

Purpose. Makes governance explicit and machine-checkable. Policies constrain behavior; they never describe it.

Attribute	Type	Required	Description
<code>rule_set</code>	text	yes	The rules, or a reference to them.
<code>rule_language</code>	string		The language <code>rule_set</code> is written in, for example "ODRL", "Rego", "DMN", "text".
<code>effect</code>	one of <code>permit</code> , <code>prohibit</code> , <code>require</code> , <code>escalate</code>		What the Policy does when it applies.
<code>trigger_conditions</code>	string-list		Conditions under which the Policy applies.
<code>severity</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		Consequence of a violation.
<code>override_protocol</code>	text		Who may override the Policy, and how.

Relationships.

- `escalates_to` → Role / Human Actor (one or more)

Notes.

- AOW does not prescribe a policy language. `rule_set` may hold rules in ODRL, Rego, Cedar, DMN, or plain text, named in `rule_language`.

4.8 Plan

Layer: Cognition · since 2.0 · `aow:Plan`

Definition. An ordered or partially ordered set of Tasks intended to fulfill an Intent, produced by an Orchestrator or other Actor.

Purpose. Separates deciding how to do something from doing it, so that plans can be inspected, approved, compared, and replayed.

Attribute	Type	Required	Description
<code>status</code>	one of <code>draft</code> , <code>approved</code> , <code>executing</code> , <code>completed</code> , <code>failed</code> , <code>abandoned</code>		Where the Plan is in its life.
<code>exception_protocol</code>	text		What to do when a Task fails or a Policy blocks progress.

Relationships.

- `constrained_by` → Policy (one or more)
- `realizes` → Intent (*required*)
- `planned_by` → Actor
- `has_task` → Task (one or more)
- `approved_by` → Human Actor (one or more)
- `has_confidence` → Confidence

Notes.

- Listed as an optional concept in version 1.0; a full entity in 2.0.
- The Orchestrator's `dependency_graph` in version 1.0 is the Plan's set of Tasks and their `depends_on` links.

4.9 Task

Layer: Execution · since 2.0 · aow:Task

Definition. A unit of assigned work within a Plan, performed by an Actor filling a Role.

Purpose. Gives "task" a single meaning. A Task is what gets assigned; an Action is what gets done; a Skill is what makes it possible.

Attribute	Type	Required	Description
<code>status</code>	one of <code>submitted</code> , <code>working</code> , <code>input-required</code> , <code>completed</code> , <code>failed</code> , <code>canceled</code> , <code>rejected</code>		Where the Task is in its life.
<code>due_by</code>	datetime		When the Task must be completed.
<code>description</code>	text	yes	Longer human-readable explanation.

Relationships.

- `shaped_by` → Context (one or more)
- `concerns` → Work Item (one or more)
- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level
- `depends_on` → Task (one or more)
- `assigned_to` → Actor
- `requires_role` → Role

Notes.

- Task states follow the Agent2Agent (A2A) protocol's task lifecycle so that AOW Tasks can be exchanged over A2A without translation.

4.10 Actor

Layer: Execution · abstract · since 2.0 · `aow:Actor`

Definition. Anything that can be assigned Tasks, perform Actions, and be held to account for them: an Agent, a person, an Orchestrator, or a Guardian.

Purpose. Lets the ontology describe work the same way whoever performs it, so that work can move between people and agents without being re-modeled.

Attribute	Type	Required	Description
name	string		Display name.

Relationships.

- constrained_by → Policy (one or more)
- fills_role → Role (one or more)
- has_skill → Skill (one or more)
- writes_to → Memory (one or more)

4.11 Agent

Layer: Execution · a kind of Actor · since 1.0 · aow:Agent

Definition. A software Actor that interprets Intents and Tasks and acts on them using Skills, within the autonomy it has been cleared for.

Purpose. The primary performer of agentic work, described by what it may do rather than by how it is built.

Attribute	Type	Required	Description
autonomy_level	→ AssuranceLevel		The highest Assurance Level the Agent has been cleared to operate at. The Agent may operate at a lower level on a given Task, never a higher one.
implementation_ref	uri		Pointer to the Agent's implementation, model card, or A2A Agent Card.

Notes.

- Deliberately says nothing about models, prompts, or frameworks. implementation_ref may point to those.

4.12 Human Actor

Layer: Execution · a kind of Actor · since 2.0 · aow:HumanActor

Definition. A person who performs, approves, reviews, or oversees work.

Purpose. Makes human-in-the-loop a first-class, traceable part of the model rather than an exception outside it.

Attribute	Type	Required	Description
<code>organizational_unit</code>	string		The team or department the person works in.

4.13 Orchestrator

Layer: Execution · a kind of Actor · since 1.0 · `aow:Orchestrator`

Definition. A coordinating Actor that turns Intents into Plans, assigns Tasks to Actors, and manages sequencing and exceptions.

Purpose. Decomposes and coordinates work so that no single Agent has to.

Attribute	Type	Required	Description
<code>planning_model</code>	string		How the Orchestrator plans, for example "rule-based", "LLM-planned", "hybrid".
<code>exception_protocol</code>	text		Default handling for failures across the Plans it runs.
<code>resource_constraints</code>	text		Limits on concurrency, cost, or time it must respect.

Notes.

- An Orchestrator may itself be an Agent, a deterministic workflow engine, or a person. The role is what matters.

4.14 Role

Layer: Execution · since 2.0 · `aow:RoLe`

Definition. A named set of responsibilities and authority that an Actor can fill, such as "claims adjuster" or "eligibility reviewer".

Purpose. Lets Tasks, Policies, and escalations target a responsibility rather than a specific person or agent, so either can fill it.

Attribute	Type	Required	Description
responsibilities	string-list		What an Actor in this Role is accountable for.
label	string	yes	Short human-readable name.

Relationships.

- constrained_by → Policy (one or more)

4.15 Skill

Layer: Execution · since 1.0 · aow:Skill

Definition. A reusable capability that an Actor can invoke.

Purpose. Describes what can be done, with its inputs, outputs, cost, and side effects, so that it can be discovered, governed, and swapped.

Attribute	Type	Required	Description
<code>input_schema</code>	json		JSON Schema (or equivalent) for the Skill's inputs.
<code>output_schema</code>	json		JSON Schema (or equivalent) for the Skill's outputs.
<code>binding</code>	string		How to invoke it, for example an MCP tool name, an API endpoint, or a UI automation.
<code>side_effects</code>	one of <code>none</code> , <code>reversible</code> , <code>irreversible</code>		Whether invoking the Skill changes external state, and whether that can be undone.
<code>latency</code>	string		Typical or maximum latency, for example "< 1s".
<code>cost_profile</code>	one of <code>low</code> , <code>medium</code> , <code>high</code>		Relative cost per invocation.

Relationships.

- `constrained_by` → Policy (one or more)
- `requires_assurance` → Assurance Level

Notes.

- `side_effects` is new in 2.0. Whether a Skill changes the world, and whether that change can be undone, is the single most important fact for deciding how much autonomy to grant around it.

4.16 Action

Layer: Execution · since 1.0 · `aow:Action`

Definition. A single execution event in which an Actor invokes a Skill, in service of a Task, at a point in time.

Purpose. The atom of accountability. Every Result traces back to an Action, and every Action to who did it, why, and under what authority.

Attribute	Type	Required	Description
<code>started_at</code>	datetime	yes	When the Action began.
<code>ended_at</code>	datetime		When the Action ended.
<code>status</code>	one of <code>succeeded</code> , <code>failed</code> , <code>blocked</code> , <code>escalated</code> , <code>compensated</code>		How the Action ended.
<code>rationale</code>	text		The Actor's stated reason for the Action, recorded for explainability.
<code>idempotency_key</code>	string		Key that makes a retried Action safe to repeat.
<code>inputs</code>	json		The inputs passed to the Skill.

Relationships.

- `concerns` → Work Item (one or more)
- `performed_by` → Actor (*required*)
- `fulfills` → Task
- `invokes` → Skill
- `approved_by` → Human Actor (one or more)
- `has_confidence` → Confidence

Notes.

- Named in version 1.0 but never defined. In 2.0 the Action, not the Skill, produces the Result: a Skill is a capability, an Action is its use.

4.17 Result

Layer: Execution · since 1.0 · `aow:Result`

Definition. The immediate, atomic effect or output of an Action.

Purpose. Records what an Action changed or produced, so that it can be verified, rolled up into Outcomes, and fed back.

Attribute	Type	Required	Description
status	one of success , partial , failure		Whether the Action achieved what it set out to.
summary	text		What the Result is, in words.
artifacts	uri-list		Documents, records, or state changes the Result produced.

Relationships.

- approved_by → Human Actor (one or more)
- generated_by → Action (*required*)
- affects → Work Item (one or more)
- has_confidence → Confidence
- contributes_to → Outcome (one or more)

Notes.

- A Result is local and immediate. An Outcome is aggregated and business-level.

4.18 Guardian

Layer: Assurance · a kind of Actor · since 1.0 · aow:Guardian

Definition. The oversight Actor that enforces Policy and Assurance Levels, monitors behavior, and escalates to people when needed.

Purpose. Separates doing the work from checking the work, so that no Actor is its own only safeguard.

Attribute	Type	Required	Description
enforcement_mode	one of preventive , detective , corrective		Whether it blocks before, detects after, or repairs after the fact.
scope	text		The Actors, Skills, or Work Items it watches.

Relationships.

- enforces → Policy / Assurance Level (one or more)
- oversees → Actor / Plan / Skill (one or more)
- escalates_to → Role / Human Actor (one or more)

Notes.

- A Guardian may be software, a person, or both. Because it is an Actor, its own interventions are Actions and leave the same trail as everyone else's.

4.19 Confidence

Layer: Assurance · since 1.0 · aow:Confidence

Definition. A model-, rule-, or human-derived measure of certainty about an Observation, a Result, or an Action.

Purpose. Lets autonomy depend on certainty: the same Agent may act alone when it is sure and escalate when it is not.

Attribute	Type	Required	Description
value	number (0–1)	yes	Certainty between 0 and 1.
source	one of model , heuristic , rule , ensemble , human		What produced the measure.
calibrated	boolean		Whether the value has been calibrated against observed accuracy.
volatility	one of low , medium , high		How much the value is expected to move as evidence arrives.

Relationships.

- compared_against → Assurance Level

4.20 Assurance Level

Layer: Assurance · since 1.0 · aow:AssuranceLevel

Definition. A governance-set level of permitted autonomy, with the human oversight and evidence it requires.

Purpose. Makes the choice between autonomy and human oversight explicit, graded, and checkable. AOW defines five levels, AL0 to AL4.

Attribute	Type	Required	Description
<code>level</code>	integer (0–4)	yes	Position on the AL0 to AL4 scale.
<code>human_role</code>	text		What people do at this level.
<code>agent_role</code>	text		What Agents may do at this level.
<code>review_protocol</code>	text		The human oversight this level requires.
<code>default_confidence_floor</code>	number (0–1)		Confidence below which an Actor operating at this level must escalate, unless a Policy sets a different floor.
<code>risk_class</code>	string		The class of risk this level is suited to.

Notes.

- Higher levels grant more autonomy and therefore demand more assurance evidence before they are granted. See the Assurance Level scale.

4.21 Outcome

Layer: Assurance · since 1.0 · `aow:Outcome`

Definition. The aggregated business impact of one or more Results, measured against an Objective.

Purpose. Closes the loop between what agents did and what the business wanted.

Attribute	Type	Required	Description
kpi_impact	json		The measured change, for example {"cycle_time_hours": {"before": 48, "after": 0.12}}.
measured_at	datetime		When the Outcome was measured.
measurement_window	string		The period the measurement covers.
time_to_value	string		How long it took for the impact to appear.
risk_profile_change	text		How the risk picture changed as a result.

Relationships.

- evaluates → Objective (*required*)

4.22 Feedback

Layer: Assurance · since 1.0 · aow:Feedback

Definition. Structured information, from people or systems, used to improve future behavior.

Purpose. Turns Outcomes, reviews, and incidents into specific, attributable changes.

Attribute	Type	Required	Description
<code>feedback_type</code>	one of <code>accuracy</code> , <code>policy</code> , <code>exception</code> , <code>preference</code> , <code>incident</code>		What the Feedback is about.
<code>issue_detected</code>	text		What went wrong or could be better.
<code>recommended_adjustments</code>	string-list		Specific changes proposed.
<code>severity</code>	one of <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>		How urgent the Feedback is.
<code>timestamp</code>	datetime		When the Feedback was given.

Relationships.

- `feedback_on` → Outcome / Result / Action / Plan (one or more) (*required*)
- `provided_by` → Actor
- `updates` → Policy / Memory / Plan / Skill / Actor / Context (one or more)

4.23 Memory

Layer: Assurance · abstract · since 1.0 · aow:Memory

Definition. Persistent knowledge accumulated from experience, used to inform future Context.

Purpose. Lets the system learn without retraining, and remember why it did what it did.

Attribute	Type	Required	Description
<code>store_ref</code>	uri		Where the memory is held.
<code>retention_policy</code>	text		How long entries are kept and when they are purged.

Notes.

- Two kinds: Operational Memory (short-lived) and Knowledge Base (long-lived).

4.24 Operational Memory

Layer: Assurance · a kind of Memory · since 1.0 · `aow:OperationalMemory`

Definition. Short-lived, execution-oriented memory scoped to a session, Task, or Work Item.

Purpose. Carries state across the steps of a single piece of work.

4.25 Knowledge Base

Layer: Assurance · a kind of Memory · since 1.0 · `aow:KnowledgeBase`

Definition. Long-lived memory of generalized patterns, rules, and precedents.

Purpose. Carries what was learned from one piece of work into the next.

5 Provenance and versioning

Version 1.0 listed "Provenance & Versioning" as an entity. On closer inspection, it is not a kind of thing that exists alongside Intents and Agents; it is something every one of them has. A Policy has a version and an author. A Skill is revised. A Result was generated by an Action at a time. Modeling provenance as a separate entity would mean every other entity pointing to one, which adds a node without adding meaning, and fails the "minimal redundancy" criterion (section 13).

AOW 2.0 therefore treats provenance as a facet that any entity may carry, expressed with the W3C PROV Ontology (PROV-O), the established standard for provenance on the web:

Attribute	Meaning	RDF property
<code>version</code>	Version of this entity definition	<code>schema:version</code>
<code>created_at</code>	When this entity or version was created	<code>prov:generatedAtTime</code>
<code>created_by</code>	The Actor responsible for it	<code>prov:wasAttributedTo</code>
<code>revision_of</code>	The earlier version this one revises	<code>prov:wasRevisionOf</code>

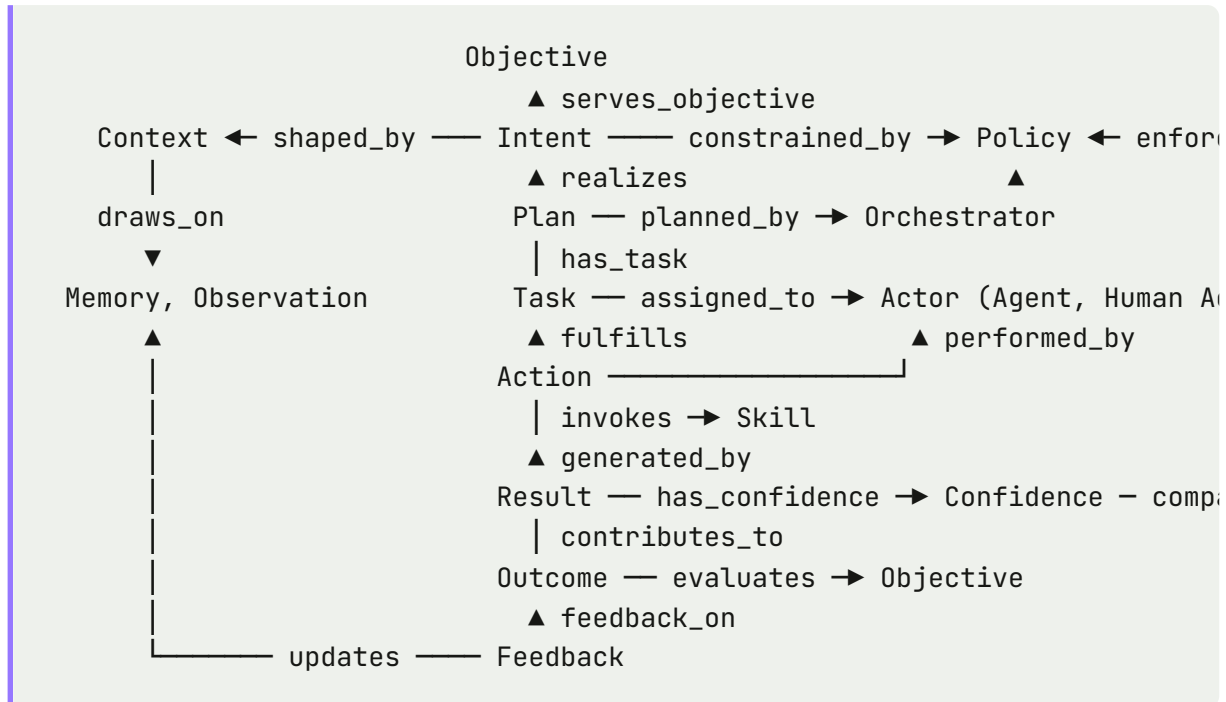
AOW's own classes and relationships are also anchored in PROV-O, so a PROV-aware tool can read an AOW graph without knowing AOW: Actor is a `prov:Agent`, Action is a `prov:Activity`, Result, Signal, and Observation are `prov:Entity`, Plan is a `prov:Plan`, `performed_by` specializes `prov:wasAssociatedWith`, `generated_by` specializes `prov:wasGeneratedBy`, and `derived_from` specializes `prov:wasDerivedFrom`.

The effect is that "who did this, when, from what, and under which version of which rule" can be answered for any entity with one vocabulary, which is the precondition for the explainability criterion in section 13.

6 The relationship model

6.1 The semantic graph

At its core, AOW is a directed, constraint-aware, learning-enabled graph that connects business goals to execution and back to learning. Its spine reads top to bottom:



Reading the spine in words:

- An **Objective** gives rise to one or more **Intents**, each a specific, actionable part of the broader goal.
- Intents are **shaped by Context** and **constrained by Policy**, and state the **Assurance Level** they require.
- An **Orchestrator** turns an Intent into a **Plan** of **Tasks**, and assigns each Task to an **Actor**, chosen by Role, Skills, and autonomy.
- Actors carry out Tasks through **Actions**, each of which **invokes a Skill** and produces a **Result**.
- Results carry a **Confidence**, which is compared against the Assurance Level to decide whether a person must confirm.
- Results **contribute to Outcomes**, which are measured against the originating Objective.
- Outcomes (and people) generate **Feedback**, which **updates** Policies, Memory, Plans, and Skills.

- **Memory** enriches future Context, so the next Intent is interpreted better than the last.
- The **Guardian** enforces Policy and Assurance Levels across all of it, and escalates to people when needed.

The published site renders this graph interactively.

6.2 All relationships

Relationship	From	To	Read backward as	Required on
serves_objective	Intent	Objective	produces intent	Intent
shaped_by	Intent, Task	Context (many)	shapes	
draws_on	Context	Memory, Observation (many)	informs	
concerns	Intent, Task, Action	Work Item (many)	is subject of	
constrained_by	Intent, Plan, Task, Actor, Skill, Role	Policy (many)	constrains	
requires_assurance	Intent, Task, Skill	Assurance Level	required by	Intent
realizes	Plan	Intent	realized by	Plan
planned_by	Plan	Actor	plans	
has_task	Plan	Task (many)	part of plan	
depends_on	Task	Task (many)	prerequisite of	
assigned_to	Task	Actor	assigned	
requires_role	Task	Role	required for	
fills_role	Actor	Role (many)	filled by	
has_skill	Actor	Skill (many)	available to	
performed_by	Action	Actor	performs	Action

Relationship	From	To	Read backward as	Required on
fulfills	Action	Task	fulfilled through	
invokes	Action	Skill	invoked by	
approved_by	Action, Plan, Result	Human Actor (many)	approves	
generated_by	Result, Signal, Observation	Action	generated	Result
derived_from	Observation	Signal (many)	source of	Observation
about	Observation	Work Item, Actor, Action, Task	observed by	
affects	Result	Work Item (many)	affected by	
has_confidence	Result, Observation, Action, Plan	Confidence	confidence of	
compared_against	Confidence	Assurance Level	threshold for	
contributes_to	Result	Outcome (many)	aggregates	
evaluates	Outcome	Objective	evaluated by	Outcome
feedback_on	Feedback	Outcome, Result, Action, Plan (many)	received feedback	Feedback
provided_by	Feedback	Actor	provided	
updates	Feedback	Policy, Memory, Plan, Skill, Actor, Context (many)	updated by	
writes_to	Actor	Memory (many)	written by	

Relationship	From	To	Read backward as	Required on
<code>enforces</code>	Guardian	Policy, Assurance Level (many)	enforced by	
<code>oversees</code>	Guardian	Actor, Plan, Skill (many)	overseen by	
<code>escalates_to</code>	Guardian, Policy	Role, Human Actor (many)	escalation target of	

6.3 Design notes

One direction per relationship. Each relationship is stated once, in the direction that lets the entity that knows about the link record it (a Result knows which Action generated it; the Action need not list every Result). Each has an inverse reading for documentation ("read backward as"), but no separate inverse property, which keeps the vocabulary small. Queries can traverse in either direction.

Tasks and Actions, not "delegates to." Version 1.0 had an Orchestrator "delegate to" Agents. In 2.0 that is expressed as a Plan the Orchestrator produced (`planned_by`) containing Tasks `assigned_to` Actors, so delegation is recorded on the Task it concerns and not duplicated.

Policies constrain; they do not act. A Policy never performs an Action. When a rule blocks or escalates something, the blocking or escalating is an Action performed by a Guardian, which leaves a trail like any other Action (see the public-sector example in section 11).

7 Assurance Levels and governed autonomy

Version 1.0 used "AL2" and "Delegated" in its examples without defining a scale. Version 2.0 defines one.

7.1 The scale

An **Assurance Level** states how much autonomy the governance function is prepared to grant for a piece of work, and therefore how much evidence and oversight that grant requires. Higher levels grant more autonomy and demand more assurance before they are granted.

Level	Name	People	Agents	Default confidence floor	Suited to
AL0	Manual	Performs the work.	Observes and prepares; takes no Action that changes a Work Item.	—	Any risk; the default for work not yet assessed.
AL1	Assisted	Decides. Approves every Action that changes a Work Item before it takes effect.	Recommends, drafts, and prepares.	—	High-risk or irreversible decisions; new or poorly characterized work.
AL2	Delegated	Confirms when confidence is low or a Policy triggers; reviews a sample otherwise.	Decides and acts within Policy.	0.90	Moderate-risk, well-characterized decisions.
AL3	Supervised	Reviews by exception, when the Guardian raises an alert, and in periodic audit.	Acts, and must be able to reverse or compensate its Actions.	0.95	Low-to-moderate risk, reversible effects.
AL4	Autonomous	Sets Policy and monitors aggregate Outcomes.	Acts without routine human review; the Guardian can suspend it at any time.	0.98	Low-risk, reversible, high-volume, well-characterized work.

The scale is published as five named individuals of the Assurance Level class, `aow:AL0` to `aow:AL4`, so documents refer to a level by its IRI (for example, `"requires_assurance": "aow:AL2"`).

The scale follows a long line of work on levels of automation, from Sheridan and Verplank's original ten-level scale (1978), through Parasuraman, Sheridan, and Wickens's model of types and levels of human interaction with automation (2000), to the SAE J3016 levels of driving automation. AOW's levels are fewer and tied to enterprise governance: each one says what the person does, what the Agent may do, and what review is required.

7.2 The effective level

Several parties have a say in how autonomous a given Action may be:

- the **Intent** states the level its work requires (`requires_assurance` , required)
- a **Task** may lower that level for one step (`requires_assurance` , optional)
- a **Skill** may cap the level at which it may be used, typically because it has irreversible side effects
- an **Agent** has been cleared to operate up to a level (`autonomy_level`)

The **effective Assurance Level** of an Action is the lowest of these. Autonomy can only be narrowed along the way, never widened: an Agent cleared for AL3 working on an Intent that requires AL2 operates at AL2.

7.3 Confidence and escalation

Within its effective level, an Agent's autonomy also depends on how sure it is. Each level from AL2 up has a default confidence floor (0.90, 0.95, 0.98). When the Confidence attached to a state-changing Action falls below the floor, the Agent must escalate, recording the Action with status `escalated` and letting a person decide. A Policy may set a different floor; the defaults exist so that "sure enough" has a number before anyone has set one.

At AL0 and AL1, every Action that changes a Work Item must be approved by a person (`approved_by`). At AL3 and above, an Agent must not use a Skill whose side effects are irreversible without approval, because the review at those levels happens after the fact.

7.4 Checking it

These rules are published as an executable query, `queries/oversight-gaps.rq` , which lists every state-changing Action by an Agent that went ahead without the oversight its effective level requires. For a conformant deployment, the answer is empty. The query runs in the project's continuous integration against every example, and a deliberately broken example (an Agent acting at 0.72 confidence under an AL2 Intent, with no approval) is kept to prove it catches the gap.

This is what "governed autonomy" (Principle 3) means in practice: not that governance exists somewhere, but that whether it was followed can be checked from the record.

8 The lifecycle

The lifecycle follows one pass of agentic work from business goal to improvement. Each stage names the entities it creates or uses.

1. **Goal setting** (Objective). The business defines Objectives, tied to KPIs, SLAs, regulatory requirements, or operational targets. *Example: reduce claims cycle time by 20%.*
2. **Intent formation** (Intent). An Intent structures the Objective into machine-interpretable form and states the Assurance Level the work requires. *Example: automate eligibility determination, at AL2.*
3. **Perception** (Work Item, Signal, Observation). Signals about the Work Item are captured and interpreted into Observations. *Example: a claim form arrives; the claim amount is extracted.*
4. **Context enrichment** (Context, Memory). Real-time data, procedure, history, and Memory enrich the Intent. *Example: claim type disability; claimant age 45; policy form LTD-2024.*
5. **Policy application** (Policy). Policies filter what is permitted, required, or prohibited. *Example: auto-approve claims under \$50,000 when confidence is at least 0.90.*
6. **Planning** (Orchestrator, Plan, Task). The Orchestrator produces a Plan of Tasks: extract claim details, validate coverage, calculate the benefit, record the decision.
7. **Assignment** (Task, Actor, Role). Each Task is assigned to an Actor that can fill the required Role and holds the necessary Skills, at an autonomy level the Task allows.
8. **Execution** (Action, Skill). Actors perform Actions, each invoking a Skill, and record their rationale.
9. **Result generation** (Result, Confidence). Each Action produces a Result, annotated with its Confidence, artifacts, and the Work Items it affected.
10. **Assurance verification** (Guardian, Assurance Level, Human Actor). The Guardian checks compliance, Assurance Levels, Policies, and provenance, and escalates to people where required. People approve what their level requires them to.
11. **Outcome assessment** (Outcome). Results aggregate into Outcomes measured against the Objective: cycle time, error rate, SLA attainment.
12. **Feedback** (Feedback). Outcomes, reviewers, and incidents produce Feedback that updates Policies, Plans, Skills, and agent behavior.
13. **Memory update** (Memory, Knowledge Base). The trace is recorded for future Context, knowledge improvement, and audit.

Version 1.0's eleven stages are all here. Perception (3) and Assignment (7) are new, because 1.0 had no entity to hang them on.

9 Formal representation

Version 1.0 said the ontology "should be implementable using JSON-LD, RDF/OWL, GraphQL schemas, and knowledge graph frameworks." Version 2.0 provides those implementations, all generated from one source file so they cannot drift apart.

9.1 Namespace and identifiers

Namespace	<code>https://w3id.org/aow#</code> (prefix <code>aow:</code>)
Ontology IRI	<code>https://w3id.org/aow</code>
This version	<code>https://w3id.org/aow/2.0.0</code>
JSON-LD context	<code>https://w3id.org/aow/context.jsonld</code>

The identifiers use w3id.org, a community-run permanent identifier service, so they will keep resolving if the project's hosting changes.

9.2 The artifacts

Artifact	Format	What it is for
<code>aow.yaml</code>	YAML	The canonical source. Everything below is generated from it.
<code>ontology/aow.ttl</code> , <code>aow.jsonld</code>	OWL 2 (Turtle, JSON-LD)	The ontology itself, for reasoners, knowledge graphs, and ontology editors.
<code>ontology/context.jsonld</code>	JSON-LD 1.1 context	Lets plain AOW JSON be read as RDF with no changes.
<code>ontology/shapes.ttl</code>	SHACL	Checks that a graph of AOW data is well formed: types, required links, allowed values.
<code>schemas/*.schema.json</code>	JSON Schema 2020-12	Checks individual AOW JSON documents, for teams that never touch RDF.
<code>queries/*.rq</code>	SPARQL	Explanation and oversight checks (sections 7.4 and 10).
<code>examples/</code>	JSON-LD	The worked example and industry examples in this paper, all valid.

9.3 One document, two readings

AOW JSON is ordinary JSON: `snake_case` keys, string identifiers, and the `type` of each entity. The JSON-LD context maps each key to its RDF property, so the same document is also RDF. An Intent from the worked example:

```

{
  "@context": ["https://w3id.org/aow/context.jsonld", { "@base": "https://",
  "@graph": [
    {
      "id": "intent-eligibility-2026",
      "type": "Intent",
      "goal_statement": "Determine eligibility for incoming disability cl
      "serves_objective": "obj-claims-cycle-time",
      "shaped_by": ["ctx-claims-procedure"],
      "constrained_by": ["policy-auto-approval", "policy-high-risk-review
      "requires_assurance": "aow:AL2",
      "priority": "high"
    }
  ]
}

```

A team that only wants JSON validates this against `schemas/intent.schema.json` and never needs to know it is also a graph. A team that runs a knowledge graph loads it as RDF and validates it with the SHACL shapes. Both are checking the same rules.

The JSON Schemas reject unknown keys, which catches typos (`goal_statment`). Extensions are still possible: any key with a namespace prefix, such as `acme:region` , is allowed (section 14).

9.4 The validation pipeline

Every example in the repository passes four gates in continuous integration:

1. **JSON Schema:** the document is well formed.
2. **SHACL:** the graph is well formed. Every Result has an Action, every Intent serves an Objective, every link points to the right kind of thing.
3. **Explainability:** every Result can be traced back to an Objective (`queries/untraced-results.rq` returns nothing).
4. **Oversight:** no Agent acted beyond the oversight its effective Assurance Level requires (`queries/oversight-gaps.rq` returns nothing).

The repository also keeps a set of deliberately broken documents, each of which must fail at a specific gate, to prove the gates catch what they claim to.

9.5 A registry

A platform can hold its AOW entities in a registry: a catalog of the Agents, Skills, Roles, Policies, and Assurance Levels in use, plus the Intents, Plans, Tasks, and Actions that reference them. Any store that can hold the JSON documents above is a valid registry; a knowledge graph that loads them as RDF is a queryable one. Version 1.0's registry sketch (its Appendix B) is superseded by the `examples/` directory, which shows complete, valid registries for six scenarios.

10 Worked example: claims processing

This is the example from version 1.0, expanded to exercise every layer, and including a claim that a person has to decide. The complete, valid document is `examples/claims-processing.jsonld`.

Objective. Reduce disability claims cycle time by 20% without increasing leakage.

Intent. Determine eligibility for incoming disability claims automatically wherever policy allows, and route the rest to an adjuster with the work already done. Requires **AL2**.

Policies. Auto-approve claims under \$50,000 when eligibility confidence is at least 0.90. Claims with fraud indicators, or from claimants over 65 with pre-existing conditions, require adjuster review.

Perception. A claim form arrives through the intake portal (a Signal). The Validation agent extracts the claim amount, \$38,200, with confidence 0.97 (an Observation about Work Item `claim-84933`).

Context. Claim type disability; claimant age 45; claim amount \$38,200; policy form LTD-2024.

Plan. The Claims orchestrator produces four Tasks, each depending on the last: extract claim metadata, validate coverage, determine the benefit, record the decision.

Actors and Skills. The Validation, Coverage, and Benefit agents fill the Eligibility processor Role and are each cleared for AL3. They use `extract_claim_fields`, `check_coverage`, `rule_validate`, `calculate_benefit`, and `record_decision`. Only `record_decision` changes anything, and its effect is reversible.

Claim 84933: decided by an agent. The decision Task requires AL2; the Validation agent is cleared for AL3; so the effective level is AL2, with a confidence floor of 0.90. The agent's eligibility confidence is 0.93, so it records the decision itself, with its rationale: *"Coverage active, benefit calculated, no high-risk indicators; claim under \$50,000 and eligibility confidence 0.93 meets the AL2 floor of 0.90."*

Claim 84951: decided by a person. For the next claim, the agent's confidence is 0.81, below the floor. It records its Action as `escalated`, with a recommendation. A claims adjuster reviews the recommendation and the attending physician's statement, and records the decision through the same `record_decision` Skill. The adjuster's Action has exactly the same shape as the agent's would have (Principle 7).

Outcome. Median turnaround falls from 48 hours to 7 minutes over the quarter, with 64% of claims processed straight through.

Feedback. The adjuster notes that claimants over 65 with pre-existing conditions are reaching review too late, after a decision has been drafted. The Feedback updates the high-risk review Policy (now version 1.1.0, a `revision_of` 1.0.0) and the claims Knowledge Base.

Explaining a decision. Running `queries/explain-result.rq` over the example reconstructs, for every Result, the chain back to the Objective:

Result	Action	Performed by	Skill	Task	Intent	Objective
Coverage active	act-coverage-84933	Coverage agent	check_coverage	Validate coverage	Automate eligibility determination	Faster disability claims
Monthly benefit \$3,180	act-benefit-84933	Benefit agent	calculate_benefit	Determine benefit	Automate eligibility determination	Faster disability claims
Eligibility approved (0.93)	act-decision-84933	Validation agent	record_decision	Record decision	Automate eligibility determination	Faster disability claims
Approved after review	act-review-84951	Claims adjuster 17	record_decision	Record decision	Automate eligibility determination	Faster disability claims

That table is what an auditor, a regulator, or a customer asking "why?" needs, and it comes from the record, not from reconstructing events after the fact.

11 Industry examples

The same entities describe work in any industry. Each example below is a complete, valid document in `examples/industry/`. Figures are illustrative.

Insurance: claims adjudication (AL2). Objective: reduce adjudication time. Intent: triage eligibility and approve low-risk claims automatically. Context: claimant age, policy details, historical exceptions. Policy: approve low-risk claims automatically; route others to an adjuster. Result: eligibility decision at 0.95 confidence. Outcome: SLA compliance up. Feedback: extend auto-approval to small water-damage claims.

Banking: KYC and AML review (AL2). Objective: improve compliance throughput. Intent: verify identity documents and flag anomalies. Perception: a passport scan (Signal) yields a machine-readable-zone check (Observation). Policy: high-risk customers need human review of every verification. Result: documents verified. Outcome: faster onboarding. Feedback: adjust an anomaly threshold in the verification Skill.

Healthcare: prior authorization (AL1). Objective: reduce patient wait time. Intent: assess medical necessity and prepare a decision. Policy: advanced imaging needs clinical validation. Because the Intent requires AL1, the agent prepares the authorization and a clinical reviewer approves it before it takes effect (`approved_by`). Outcome: faster approvals. Feedback: update guidelines for repeat imaging.

Public sector: benefits determination (AL2). Objective: reduce backlog. Intent: check eligibility of routine applications. Policy: sensitive cases go to a specialist. When an application is flagged sensitive, the Guardian blocks the agent's Action and escalates; a specialist caseworker records the determination. All three Actions are in the record. Outcome: backlog down. Feedback: tune an income threshold.

Retail: supply chain exceptions (AL3). Objective: reduce stockouts. Intent: detect and resolve shipment anomalies. Perception: a carrier feed reports a 30-hour delay. Policy: auto-correct only when risk is under threshold and the correction can be reversed. The agent re-routes the shipment at 0.97 confidence, above the AL3 floor, with an idempotency key so a retry cannot double-book. Outcome: fewer lost sales. Feedback: feed delay patterns into forecasting.

12 How AOW relates to existing work

AOW is deliberately built from parts the industry has already validated. This section says where each comes from, and where AOW differs.

12.1 Service-Oriented Architecture

SOA is AOW's closest precedent, in purpose if not in content. SOA also had an ontology: The Open Group's SOA Ontology, later standardized as ISO/IEC 18384-3:2016, formalized services, contracts, and compositions so that architectures could be described consistently.

SOA concept	AOW equivalent	Key difference
Service	Agent	Agents reason, plan, and act within granted autonomy; services execute fixed operations.
Operation	Skill	Skills carry side effects, cost, and governance metadata.
Service contract	Skill schemas + Policy + Assurance Level	AOW adds confidence and risk to the contract.
Orchestration	Orchestrator and Plan	Plans are produced dynamically, under Context.
Registry (UDDI)	AOW registry (section 9.5)	Records learning and Outcomes, not only endpoints.

AOW extends SOA from static invocation to contextual, adaptive, governed execution.

A companion project, the [SOA-to-Agentic AI Terminology Mapping](#) (Skan, Inc., 2026), maps twenty-eight SOA terms to agentic equivalents. `crosswalks/soa-to-agentic-terms.csv` maps each of those agentic terms to the AOW class or property that formalizes it, so the two can be used together: the mapping for vocabulary, AOW for structure.

12.2 BPMN and DMN

The Object Management Group's Business Process Model and Notation (BPMN) and Decision Model and Notation (DMN) both describe work, and AOW borrows freely from them. The differences are of emphasis:

- BPMN assumes flows are designed in advance; AOW supports Plans produced at run time, under Context.

- BPMN models tasks, not reasoning; AOW models the Objectives, Context, Policies, and Outcomes around the tasks.
- BPMN is flow-first; AOW is meaning-first.

They combine well. A BPMN process can be the source of an AOW Plan, and a DMN decision table can be the `rule_set` of an AOW Policy.

12.3 Multi-agent systems research

AOW's Agents act autonomously, its Skills correspond to the capabilities of multi-agent systems literature, and its Orchestrator is a coordination mechanism. Its cognitive vocabulary echoes the belief-desire-intention (BDI) model (Rao and Georgeff, 1995): Objectives play the part of desires, Intents of intentions, and Context and Memory of beliefs. The FIPA agent communication standards are an earlier attempt at interoperable agent semantics.

What multi-agent systems research rarely includes, and AOW adds, is enterprise governance: Assurance Levels, Policies, the Guardian, provenance, and the systematic treatment of Outcomes, Feedback, and Memory.

12.4 Knowledge graphs and W3C standards

AOW can be implemented on a knowledge graph but is not one: a knowledge graph is a storage substrate; AOW is the schema for one kind of knowledge. Where the W3C has already standardized a concept, AOW reuses or aligns with it rather than reinventing it:

Standard	Used for
PROV-O	Provenance facet; Actor, Action, Result, Plan anchored in PROV classes (section 5)
Organization Ontology (ORG)	Role aligns with <code>org:Role</code>
SOSA/SSN	Observation aligns with <code>sosa:Observation</code>
ODRL 2.2	Policy aligns with <code>odrl:Policy</code> ; ODRL is one possible policy language
schema.org	Action aligns with <code>schema:Action</code>
OWL 2, SHACL, JSON-LD 1.1	The ontology's own formats

12.5 Process mining

The Perception layer is where AOW meets process mining, the discipline of reconstructing how work actually happens from the event data systems leave behind (van der Aalst, 2016). A Signal corresponds to an event in the IEEE 1849 (XES) event log standard; a Work Item corresponds to a case, or to an object in the Object-Centric Event Log (OCEL 2.0) format. Because an agent's Actions can be linked to the Signals they leave (`generated_by`), work done by agents can be mined, compared, and conformance-checked with the same methods used for work done by people and systems, which is how "the agent did what the plan said" can be verified rather than assumed.

12.6 Agent protocols: MCP and A2A

Two open protocols now carry much of the traffic between agents and their tools. AOW is designed to describe what they carry:

- **Model Context Protocol (MCP)** exposes tools and resources to models. An MCP tool is an AOW Skill (its `binding` can name it), and an MCP resource is a source of Context.
- **Agent2Agent (A2A)** lets agents delegate work to one another. A2A's Agent Card describes an AOW Agent, its skills are AOW Skills, and its Tasks are AOW Tasks. AOW's Task states use the A2A task lifecycle's states, so a Task can be carried over A2A without translation.

Both protocols are now stewarded under the Linux Foundation. AOW adds what the protocols intentionally leave out: why the work is being done (Objective, Intent), what rules apply (Policy, Assurance Level), and what came of it (Outcome, Feedback).

12.7 Observability: OpenTelemetry

The OpenTelemetry semantic conventions for generative AI, in development at the time of writing, define spans for agent operations such as invoking an agent and executing a tool, and attributes that identify agents and tools. An `execute_tool` span is a natural source for an AOW Action, its trace for the Signals the Action leaves behind, and its agent attributes for the performing Agent. AOW supplies the business meaning a trace lacks: which Task, Intent, and Objective the span was in service of.

12.8 AI governance frameworks

AOW is not a compliance framework, but it is designed to make compliance evidence easy to produce:

- **NIST AI Risk Management Framework (AI RMF 1.0).** The framework's four functions (Govern, Map, Measure, Manage) correspond to AOW's Policy and Assurance Levels, Context and Intents, Confidence and Outcomes, and Guardian and Feedback respectively.
- **ISO/IEC 42001:2023** (AI management systems) asks organizations to define roles, controls, and records for AI; AOW's Role, Policy, Guardian, and provenance facet are a vocabulary for those records.
- **EU AI Act (Regulation (EU) 2024/1689).** Article 12 requires high-risk systems to keep logs, and Article 14 requires effective human oversight. AOW's Action trail and provenance facet address the first; its Assurance Levels, `approved_by`, and oversight check address the second. Whether a given deployment complies remains a legal question AOW cannot answer for it.

13 Validation criteria

A mature ontology must meet the criteria below. Version 1.0 stated them; version 2.0 says how each is checked.

Criterion	What it requires	How AOW 2.0 checks it
Semantic completeness	Every necessary concept is represented, none is overloaded, each has a clear purpose.	Every class has a definition and a purpose (checked in CI). The six new entities close the gaps 1.0 left.
Minimal redundancy	No two entities serve the same role.	Explicit disjointness axioms (Result vs. Outcome, Signal vs. Observation, Task vs. Action, Skill vs. Action, Confidence vs. Assurance Level, Operational Memory vs. Knowledge Base, Agent vs. Human Actor). Provenance became a facet rather than a redundant entity.
Orthogonality	Entities serve independent functions.	Policies constrain; they never act (Guardian Actions do). Skills are capabilities; Actions are their use.
Extensibility	New entities can be added without breaking the structure.	Extension rules in section 14; namespaced extension keys are accepted by the JSON Schemas.
Formal mapability	Implementable in JSON-LD, RDF/OWL, and knowledge graph frameworks.	Published in OWL, JSON-LD, SHACL, and JSON Schema (section 9).
Governance-ready	Expresses versioning, provenance, auditability, risk tiering, and compliance constraints.	Provenance facet (PROV-O); Assurance Level scale; Policy; Guardian.
Execution-ready	Supports dynamic planning, multi-agent cooperation, and real-time context.	Plan, Task, and Orchestrator; Task states aligned with A2A; Context with validity.

Criterion	What it requires	How AOW 2.0 checks it
Explainability-ready	Intent to Action, Action to Result, and Result to Outcome must be reconstructable deterministically.	<code>queries/explain-result.rq</code> reconstructs the chain; <code>queries/untraced-results.rq</code> must return nothing for every example.

14 Extending AOW

AOW is meant to be extended for particular industries and platforms. To keep extensions from fragmenting it:

1. **Subclass rather than redefine.** A "Claims Agent" is an `aow:Agent` with extra attributes, not a new kind of actor.
2. **Use your own namespace.** Extension classes and properties live in your namespace, not `aow:`. In JSON, extension keys carry a prefix (`"acme:region": "EMEA"`), which the AOW JSON Schemas accept and the JSON-LD context lets you map.
3. **Align, then add.** Before adding a concept, check whether an AOW class plus a Policy, Context variable, or Skill attribute already expresses it.
4. **Propose what generalizes.** If an extension would be useful across organizations, propose it for the core ontology through the project's issue tracker.

Candidate extensions already under discussion include Simulation (testing Plans before execution), Worklet (reusable Plan fragments), and Cost (accounting for the resources an Action consumes).

15 Limitations and open questions

AOW 2.0 is a step, not an end state. Known limits:

- **Multi-party work.** AOW assumes one organization's governance. How Policies, Assurance Levels, and trust compose when agents from different organizations collaborate is open.
- **Negotiation and delegation between agents.** AOW records that a Task was assigned; it does not model the negotiation that led to the assignment.
- **Identity and authorization.** Which credentials an Agent acts under is left to identity standards; AOW records only who performed an Action.
- **Calibration.** The confidence floors assume Confidence values are calibrated. Many are not. `calibrated` records whether they are; it cannot make them so.
- **Policy semantics.** Because AOW does not prescribe a policy language, it cannot check what a Policy says, only that it exists and was applied.
- **Empirical validation.** The examples here are illustrative. The most useful next contributions are real deployments modeled in AOW, including the places it did not fit.

Disagreement with any of the modeling choices in this paper is welcome, and most useful as an issue or pull request with a concrete case attached.

16 Glossary

Term	Definition
Action	A single execution event in which an Actor invokes a Skill, in service of a Task, at a point in time.
Actor	Anything that can be assigned Tasks, perform Actions, and be held to account for them: an Agent, a person, an Orchestrator, or a Guardian.
Agent	A software Actor that interprets Intents and Tasks and acts on them using Skills, within the autonomy it has been cleared for.
Assurance Level	A governance-set level of permitted autonomy, with the human oversight and evidence it requires.
Assurance Level scale	The five levels of permitted autonomy, AL0 (Manual) to AL4 (Autonomous).
Confidence	A model-, rule-, or human-derived measure of certainty about an Observation, a Result, or an Action.
Context	The structured situational information that gives relevance to an Intent.
Effective Assurance Level	The lowest of the levels set on the Intent, Task, and Skill, and the performing Agent's autonomy_level.
Feedback	Structured information, from people or systems, used to improve future behavior.
Guardian	The oversight Actor that enforces Policy and Assurance Levels, monitors behavior, and escalates to people when needed.
Human Actor	A person who performs, approves, reviews, or oversees work.
Intent	A structured, actionable goal derived from an Objective and interpretable by agents, orchestrators, and people.
Knowledge Base	Long-lived memory of generalized patterns, rules, and precedents.
Memory	Persistent knowledge accumulated from experience, used to inform future Context.
Objective	A business-level goal expressed in strategic or operational terms.

Term	Definition
Observation	An interpreted fact about a Work Item, Actor, or Action, derived from one or more Signals and carrying a Confidence.
Operational Memory	Short-lived, execution-oriented memory scoped to a session, Task, or Work Item.
Orchestrator	A coordinating Actor that turns Intents into Plans, assigns Tasks to Actors, and manages sequencing and exceptions.
Outcome	The aggregated business impact of one or more Results, measured against an Objective.
Plan	An ordered or partially ordered set of Tasks intended to fulfill an Intent, produced by an Orchestrator or other Actor.
Policy	Declarative constraints defining permissible, required, or restricted behavior.
Provenance facet	The attributes every entity may carry to record who created it, when, from what, and which version it revises, expressed with W3C PROV-O.
Result	The immediate, atomic effect or output of an Action.
Role	A named set of responsibilities and authority that an Actor can fill, such as "claims adjuster" or "eligibility reviewer".
Signal	A raw, time-stamped trace emitted by a system, a person, or the environment: an event, a log line, a user-interface interaction, a document arriving, a state change, a metric sample.
Skill	A reusable capability that an Actor can invoke.
Task	A unit of assigned work within a Plan, performed by an Actor filling a Role.
Work Item	The unit of business work that the rest of the ontology is about: a claim, an application, an order, a case, a ticket.

17 Summary

AOW is a platform-agnostic, formally specified ontology of how intelligent work is structured, governed, executed, and learned from. What sets it apart:

- a clear separation of Objective, Intent, Plan, Task, Action, Result, and Outcome
- people and agents as interchangeable Actors, with human oversight recorded rather than assumed
- governance built into the model: Policy, a defined Assurance Level scale, Confidence floors, and an independent Guardian
- a full feedback loop into Memory
- alignment with the standards and protocols enterprises already use, from PROV-O and BPMN to MCP, A2A, and OpenTelemetry
- a machine-readable release in which explainability and oversight are checked, not just claimed

It is offered as a candidate reference model for the agentic enterprise: a starting point for shared vocabulary and, with enough use and criticism, for a standard.

Appendix A: Relationship matrix

For each entity, the kinds of entity it can point to and be pointed to by (inherited relationships included).

Entity	Points to	Pointed to by
Work Item	—	Action, Intent, Observation, Result, Task
Signal	Action	Observation
Observation	Action, Actor, Confidence, Signal, Task, Work Item	Context
Objective	—	Intent, Outcome
Intent	Assurance Level, Context, Objective, Policy, Work Item	Plan
Context	Memory, Observation	Feedback, Intent, Task
Policy	Human Actor, Role	Actor, Feedback, Guardian, Intent, Plan, Role, Skill, Task
Plan	Actor, Confidence, Human Actor, Intent, Policy, Task	Feedback, Guardian
Task	Actor, Assurance Level, Context, Policy, Role, Task, Work Item	Action, Observation, Plan, Task
Agent	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Human Actor	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Policy, Result, Task
Orchestrator	Memory, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Guardian	Actor, Assurance Level, Human Actor, Memory, Plan, Policy, Role, Skill	Action, Feedback, Guardian, Observation, Plan, Task
Role	Policy	Actor, Guardian, Policy, Task
Skill	Assurance Level, Policy	Action, Actor, Feedback, Guardian
Action	Actor, Confidence, Human Actor, Skill, Task, Work Item	Feedback, Observation, Result, Signal

Entity	Points to	Pointed to by
Result	Action, Confidence, Human Actor, Outcome, Work Item	Feedback
Confidence	Assurance Level	Action, Observation, Plan, Result
Assurance Level	—	Confidence, Guardian, Intent, Skill, Task
Outcome	Objective	Feedback, Result
Feedback	Action, Actor, Context, Memory, Outcome, Plan, Policy, Result, Skill	—
Operational Memory	—	Actor, Context, Feedback
Knowledge Base	—	Actor, Context, Feedback

Appendix B: Changes from version 1.0

Area	Version 1.0	Version 2.0
Entities	15 described, 17 in the registry	25 classes, 8 of them new: Work Item, Signal, Observation, Plan, Task, Role, Actor, Human Actor. Orchestrator and Guardian are now kinds of Actor
Action	In glossary and registry; no definition	Defined; produces Results
Result	Produced by Skill (section 3.8) or Action (glossary)	Produced by Action
Plan	"Optional intermediate concept"	Full entity; realizes an Intent
Task	Used throughout; undefined	Defined; states follow A2A
People	Implicit ("human review")	Human Actor and Role; <code>approved_by</code>
Autonomy	"AL2," "Delegated" used, undefined	AL0 to AL4 scale; effective-level rule; confidence floors
Provenance & Versioning	Two registry entities without attributes	A facet of every entity, expressed with PROV-O
Guardian	Definition only	Actor with attributes and relationships; its interventions are Actions
Memory	Definition and two types	Abstract class with two subclasses and attributes
Assurance Level	No relationships	Referenced by Intent, Task, Skill, Agent, Confidence, Guardian
Skill	No side-effect information	<code>side_effects</code> : none, reversible, irreversible
Identifiers	<code>intent_id</code> , <code>agent_id</code> , and so on	<code>id</code> on every entity; <code>type</code> names the class

Area	Version 1.0	Version 2.0
Agent	<code>skills</code> , <code>memory_link</code> , <code>role</code> , <code>assurance_level_required</code> attributes	<code>has_skill</code> , <code>writes_to</code> , <code>fills_role</code> relationships; required assurance moved to Intent, Task, and Skill
Machine-readable form	Described	OWL, JSON-LD, SHACL, JSON Schema, SPARQL, validated examples
Validation criteria	Stated	Each paired with a check
Relation to other work	SOA, BPMN, MAS, knowledge graphs	Adds PROV-O and W3C standards, process mining, MCP, A2A, OpenTelemetry, NIST AI RMF, ISO/IEC 42001, EU AI Act
Editorial	Mixed section numbering; version referred to as "v3" in section 11; example YAML lost its indentation	Corrected throughout

References

- Berti, A., et al. (2024). *OCEL (Object-Centric Event Log) 2.0 Specification*. arXiv:2403.01975.
- Erl, T. (2005). *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall.
- European Union (2024). Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*.
- FIPA (2002). *FIPA ACL Message Structure Specification* (SC00061G). Foundation for Intelligent Physical Agents.
- Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
- IEEE (2016). *IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams* (IEEE 1849).
- ISO/IEC (2016). *ISO/IEC 18384-3:2016 Information technology — Reference Architecture for Service Oriented Architecture (SOA RA) — Part 3: Service Ontology*.
- ISO/IEC (2023). *ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system*.
- Linux Foundation. *Agent2Agent (A2A) Protocol Specification*. <https://a2a-protocol.org>
- Model Context Protocol. *Model Context Protocol Specification*. <https://modelcontextprotocol.io>
- NIST (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology.
- Object Management Group (2013). *Business Process Model and Notation (BPMN), Version 2.0.2*.
- Object Management Group. *Decision Model and Notation (DMN)*.
- OpenTelemetry. *Semantic Conventions for Generative AI Systems*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- Parasuraman, R., Sheridan, T. B., and Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, 30(3), 286–297.
- Rao, A. S., and Georgeff, M. P. (1995). BDI agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, 312–319.
- SAE International (2021). *J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*.

- Sheridan, T. B., and Verplank, W. L. (1978). *Human and Computer Control of Undersea Teleoperators*. MIT Man-Machine Systems Laboratory.
- Skan, Inc. (2026). *SOA-to-Agentive AI Terminology Mapping*, version 2.0.0. Zenodo. <https://doi.org/10.5281/zenodo.21823088>
- The Open Group. *Service-Oriented Architecture Ontology*, Technical Standard.
- van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
- W3C (2012). *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation.
- W3C (2013). *PROV-O: The PROV Ontology*. W3C Recommendation.
- W3C (2014). *The Organization Ontology*. W3C Recommendation.
- W3C (2017). *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- W3C (2017). *Semantic Sensor Network Ontology*. W3C Recommendation.
- W3C (2018). *ODRL Information Model 2.2*. W3C Recommendation.
- W3C (2020). *JSON-LD 1.1*. W3C Recommendation.
- Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley.